Truong Phi Hung - Project Portfolio Page

Overview

Keyboard Warrior is a Java terminal game that turns typing practice into combat: players choose skills and type passages against a countdown, with speed and accuracy determining attack damage. A complete run consists of five scenarios offering combat, puzzle, reaction or dialogue encounters, followed by a two-phase final boss. Players can choose a difficulty and seed, continue a saved run, and view their typing history and all-time best attempt.

Summary of Contributions

Code contributed: tP Code Dashboard - vanitasluna.

Enhancements implemented

- **Typing-result evaluation and feedback** (PR #6): Implemented the evaluator, result model, WPM calculation, error reporting and result formatting. Separated calculation from presentation so gameplay can show a concise summary while retaining detailed word errors for other uses.
- **Character accuracy and damage calculation** (PR #24): Replaced positional word accuracy with Levenshtein distance over Unicode code points, so an insertion or deletion does not make all later words appear incorrect. The implementation stores two dynamic-programming rows instead of the full distance matrix. Changed baseline Hard-mode damage to `floor(WPM * (accuracyPercent / 100)^4)` and removed the separate timeout penalty while retaining the countdown and automatic submission. At 50 WPM and 95% accuracy, an attack deals 40 damage.
- **Combat balance and two-phase final boss** (PR #24): Balanced regular enemy health and added the Kernel Overlord after scenario five. Implemented phase transitions that preserve player health, discard excess damage and grant no extra enemy attack. Victory requires defeating both phases. Balance simulations check that 50 WPM and at least 95% accuracy can complete a run against each repeated regular enemy type.
- **Playable run and terminal input** (PR #12, PR #13): Integrated teammates' state, timing and combat components into the playable loop. Used JLine to display the countdown and capture unfinished input at expiry. Added terminal capability checks and Windows/macOS/Linux launchers to address redirected input in Gradle and IDE consoles.
- **Standalone JAR startup and OS smoke-test recovery** (PR #35): Fixed startup refusal in redirected or unsupported consoles by providing plain-text input. Kept live countdowns in capable terminals, enabled native access in the executable JAR manifest, and added packaged-JAR tests for startup, EOF, invalid commands and typing submission.

Contributions to documentation

- **User Guide:** Wrote and updated Quick Start, Skills and countdown, Combat and progression and Typing feedback. Documented the teammate's saved-history feature after integration.
- **Developer Guide:** Documented terminal input, typing evaluation and damage, scenario progression and the final boss, design alternatives and testing. Added the typing-evaluation sequence diagram, run-progression class diagram and boss-phase-transition sequence diagram, with editable PlantUML sources.
- **DG review feedback** (PR #51): Addressed TA issues #42-47 by adding architecture, runtime, state, storage and object views; correcting static participants, relationship labels, nested activations and call order; shortening repeated explanations; and regenerating diagrams and the DG PDF.

Testing and team-based tasks

- Added evaluator and gameplay regression tests covering character edits, Unicode input, timeout submission, enemy defeat, boss transitions and completion of a balanced run.
- Resolved integration defects after the saved-history merge (PR #25): retained the same history store across menu/gameplay transitions, isolated tests from real saved records and fixed locale-dependent enemy lookup. Synchronized source, tests and documentation while preserving teammates' existing storage behavior.
- For final-release integration, connected teammates' event encounters to the playable run and extended saves to retain selected encounters, earned skills and countdown adjustments. Added integration checks for event progression, timeout handling and continuing mixed-encounter runs.